
Яндекс

Яндекс Лицей

Прерывание цикла

Как спастись из матрёшки циклов и пошпионить за кодом

Что впереди

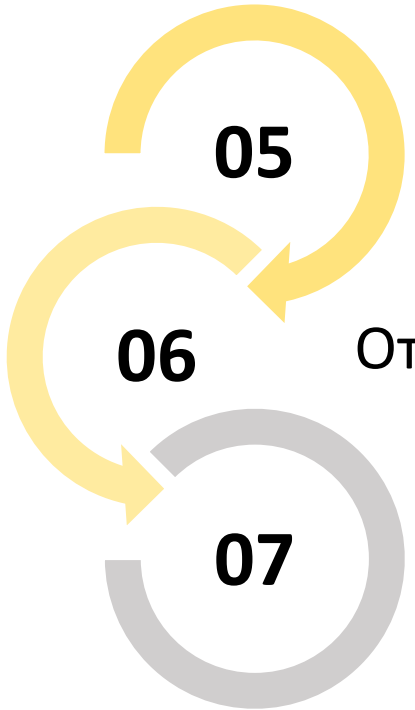


Переменные как флаги
Булевы значения переменных

Прерывание цикла
Как прервать итерацию или цикл

Else-часть у циклов
Если не вышло, то что?

Вложенные циклы
Можно выполнять цикл, пока выполняешь цикл



Выход из цикла
Как выйти с любой глубины

Отладка программ
Дебаггинг

Итоги



Поднять булев флаг, или

Переменные как флаги

Переменные как флаги



Переменная, которая содержит булево значение, называется флагом. В примере флажок — булева переменная was_pressed, которая сначала False (кнопка не нажата), а если кнопку нажали, становится True. Всё. Это флаг.

```
was_pressed = False
if button_pressed:
    was_pressed = True
```

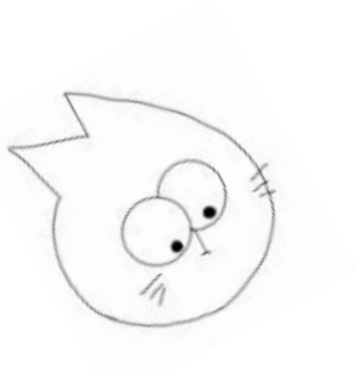
Чаще всего схема с флагами используется в комбинации с циклом while, где флажок является единственным условием завершения цикла:

```
ready_to_exit = False
while not ready_to_exit:
    command = input()
    if command == 'exit':
        ready_to_exit = True
    ... # Какой-то полезный код далее.
```

Либо является дополнительным «пожарным выходом»:

```
need_emergency_exit = False
check_sum = 0
# Запасной выход должен быть записан через and
                с основным условием.
while check_sum < 1000 and not need_emergency_exit:
    command = input()
    if command == 'exit':
        need_emergency_exit = True
    ... # Какой-то полезный код далее.
```

Переменные как флаги



Пример флага с циклом for

```
forbidden_word = 'синхрофазотрон'
print('Введите десять слов, но постарайтесь случайно не ввести слово "' +
    forbidden_word + '"!')
# Можно было использовать и sep='', чтобы кавычки не отклеились от слова.
said_forbidden_word = False
for i in range(10):
    if said_forbidden_word:
        print('Напоминаем, будьте осторожнее, не введите снова слово "' +
            forbidden_word + '"!')
    word = input()
    if word == forbidden_word:
        said_forbidden_word = True  # Но цикл дойдём до конца.
    # Вместо предыдущих двух строк также можно написать:
    # said_forbidden_word = (said_forbidden_word or word == forbidden_word)
if said_forbidden_word:
    print('Вы нарушили инструкции.')
else:
    print('Спасибо, что ни разу не упомянули', forbidden_word)
```



Прерывание цикла

Выходим красиво

Прерывание цикла

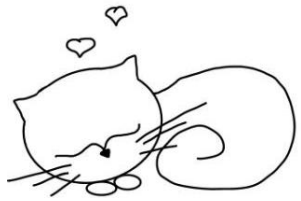
Оператор break — это ваш личный телепорт из цикла. Он мгновенно переносит вас за его пределы, даже если итерации не закончились.

Пример с while:

```
while True:
    action = input('Что делать? (стоп/продолжить): ')
    if action == 'стоп':
        print('Сбежали!')
        break
    print('Делаем:', action)
print('Бесконечный цикл дошёл до конца')
```

Пример с for:

```
for num in 1, 3, 5, 8, 9, 7:
    if num % 2 == 0:
        print(f"Первое чётное: {num}")
        break
print(f"Закончили цикл на числе {num}")
```



Прерывание цикла

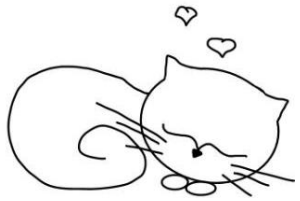
Одну и ту же программу можно написать с break, а можно с флагом. Сравните:

С флагом:

```
count = 0
need_exit = False
while not need_exit:
    command = input("Введите 'e' для выхода: ")
    if command == 'e':
        need_exit = True
    else:
        count += 1
        print(f"Вы зашли в цикл {count} раз.")
```

С break:

```
count = 0
while True:
    command = input("Введите 'e' для выхода: ")
    if command == 'e':
        break
    count += 1
    print(f"Вы зашли в цикл {count} раз.")
```





Прерывание итерации, или

Эта итерация нам не идёт

Эта итерация нам не идёт



Оператор continue — это как сказать «пока-пока» текущей итерации и сразу перепрыгнуть к заголовку цикла (для проверки условия или для получения следующего элемента последовательности).

```
for i in range(6):
    if i == 2:
        # Испытываем личную неприязнь к двойкам.
        continue
    print(i)
```

```
count = 1
while count < 100:
    if count % 5 == 0:
        continue
    print(count)
    count += 1
```

Используйте continue, чтобы избегать вложенных if. Код становится чище:

```
# Плохо
for item in items:
    if условие:
        # 10 строк кода
```

```
# Хорошо
for item in items:
    if не_условие:
        continue
    # 10 строк кода
```



Необязательная часть else, или А иначе что?

А иначе что?



Блок else у цикла выполняется, если он завершился без break.

Пример с while:

```
password = "secret"
attempts = 3
while attempts > 0:
    guess = input("Введите пароль: ")
    if guess == password:
        print("Доступ разрешён!")
        break
    attempts -= 1
else:  # Если закончились попытки.
    print("Доступ запрещён. Вызываю полицию!")
```

Пример с for:

```
print('Предъявите число, я проверю, простое ли оно.')
num = int(input())
for i in range(2, num):
    if num % i == 0:
        print(f"{num} не простое.")
        break
else:
    # Сработает, если делителей нет.
    print(f"{num} простое!")
```



Вложенные циклы, или

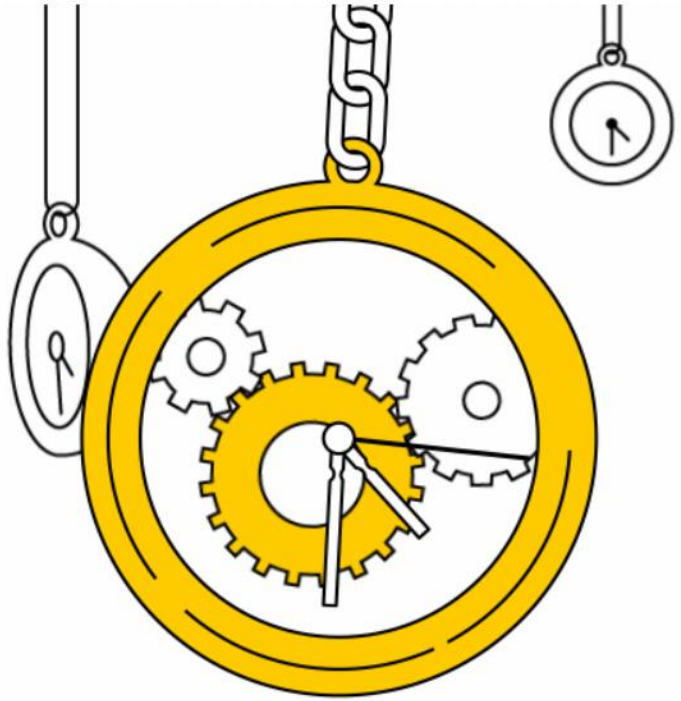
Можно выполнять цикл, пока выполняешь цикл

Вложенные циклы



В Python можно бесконечно вкладывать одни блочные операторы в другие.

```
for h in range(24):  # Одни сутки — 24 часа.
    for m in range(60):  # В каждом часе 60 минут.
        for s in range(60):  # В каждой минуте 60 секунд.
            print(f"Итерация внутреннего цикла: {(h + 1) * (m + 1) * (s + 1)}")
            print(f"Время: {h:02d}:{m:02d}:{s:02d}")
```



Тот цикл, который находится на более верхнем уровне (с меньшим уровнем вложенности), называют **внешним**, а тот, что внутри, — **вложенным**.
Каждую итерацию внешнего цикла внутренний выполняется полностью.

Вложенные циклы



Ещё пример. Пусть необходимо вывести в строку таблицу умножения для заданного числа. Задача решается так:

```
k = int(input())
for i in range(1, 10):
    print(f"{i} * {k} = {k * i}", end='\t')
```

А если нам нужно вывести таблицу умножения для всех чисел от 1 до k?

```
k = int(input())
for j in range(1, k + 1):
    for i in range(1, 10):
        print(f"{i} * {j} = {j * i}", end='\t')
    print()  # Пустая строка после
    всех итераций внутреннего цикла.
```

```
1 * 1 = 1   2 * 1 = 2   3 * 1 = 3   4 * 1 = 4   5 * 1 = 5   6 * 1 = 6   7 * 1 = 7   8 * 1 = 8   9 * 1 = 9
1 * 2 = 2   2 * 2 = 4   3 * 2 = 6   4 * 2 = 8   5 * 2 = 10  6 * 2 = 12  7 * 2 = 14  8 * 2 = 16  9 * 2 = 18
1 * 3 = 3   2 * 3 = 6   3 * 3 = 9   4 * 3 = 12  5 * 3 = 15  6 * 3 = 18  7 * 3 = 21  8 * 3 = 24  9 * 3 = 27
1 * 4 = 4   2 * 4 = 8   3 * 4 = 12  4 * 4 = 16  5 * 4 = 20  6 * 4 = 24  7 * 4 = 28  8 * 4 = 32  9 * 4 = 36
1 * 5 = 5   2 * 5 = 10  3 * 5 = 15  4 * 5 = 20  5 * 5 = 25  6 * 5 = 30  7 * 5 = 35  8 * 5 = 40  9 * 5 = 45
```

Катапультирование из любой глубины



Операторы break и continue классные, но они работают только непосредственно в том цикле, в котором находятся. Но что делать, если нам надо вырваться из второго уровня вложенности?

Один break хорошо, а в комбинации с флагом — ещё лучше:

```
print("Наша программа боится чисел больше 700.")
runaway = False
for j in range(1, 100):
    for i in range(1, 100):
        if i * j > 700:
            print('A-a-a-a-a-a, попалось слишком большое число!')
            runaway = True  # Ставим флаг.
            break  # Выходим из вложенного цикла.
        print(i ** j)
    if runaway:
        break  # Выходим из внешнего цикла.
```



Отладка программы, или

Как искать чёрную кошку в тёмной комнате, а потом понять, что это был енот

Отладка (дебаггинг)



Рассмотрим пример. Нам надо написать программу, которая заставляет пользователя вводить числа до тех пор, пока он не введёт число-палиндром. Делать мы это будем через сбор зеркального числа с помощью операции получения остатка от деления на 10.

```
print('Тренажёр по вводу палиндрома:')
while True:
    print('Введите число-палиндром:')
    number = n = int(input())
    reverse = 0
    while n > 0:
        reverse = reverse * 10 + n % 10
        n //= 10
    if number == reverse:
        print('Вы ввели палиндром! Программа остановлена.')
        break
    print('Введённое число не палиндром, попробуйте ещё раз.')
```

Самый базовый путь «отладки» — навтыкать print() в стратегические места

```
while n > 0:
    reverse = reverse * 10 + n % 10
    n //= 10
    print(f"n: {n}, reverse: {reverse} → Может, уже сдадитесь, палиндромы?")  # Шпионим!
```

Отладка по-взрослому



Инструкция:

1. Ставим ловушки (точки останова или брейкпоинты) в потенциально интересных местах. Для этого надо кликнуть слева от номера строки, чтобы появился красный кружок — обозначение точки останова:

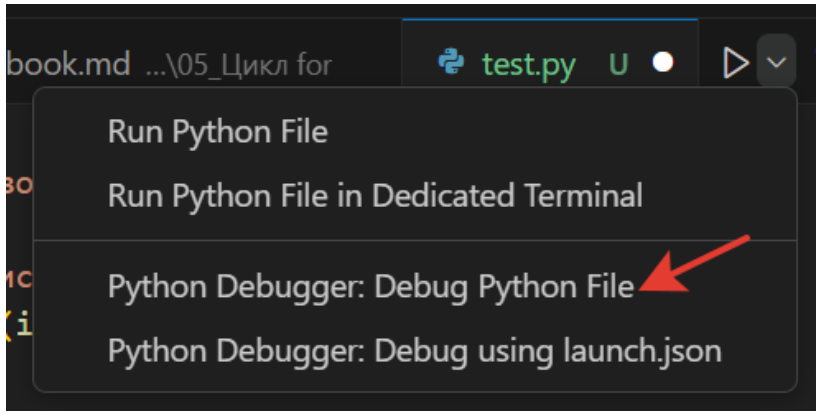
- На строке number = n = int(input()) → чтобы поймать момент ввода.
- Внутри внутреннего цикла → следим, как reverse растёт, как гриб после дождя.
- На if number == reverse → проверяем финальный аккорд.

```
1   print('Тренажер по вводу палиндрома:')
2   while True:
3       print('Введите число палиндром:')
4       number = n = int(input())
5       reverse = 0
6       while n > 0:
7           reverse = reverse * 10 + n % 10
8           n //= 10
9       if number == reverse:
10          print('Вы ввели палиндром! Программа остановлена.')
11          break
12      print('Введенное число не палиндром, попробуйте еще раз.')
```

Отладка по-взрослому

Инструкция:

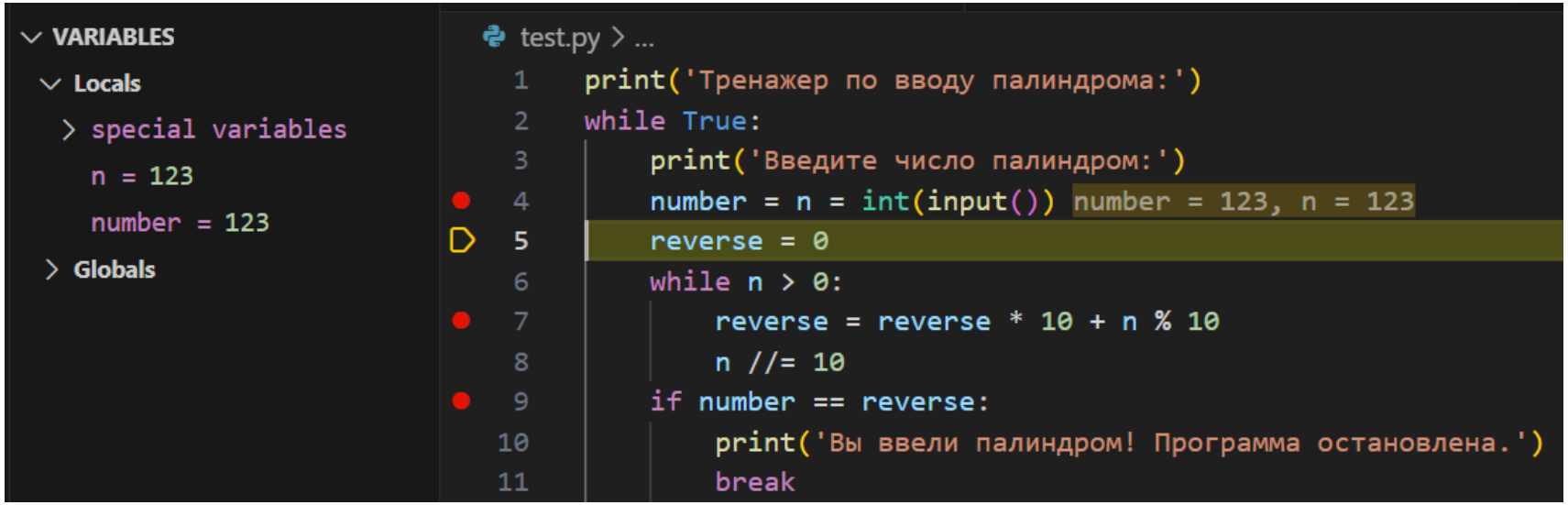
2. Запускаем дебаггер.



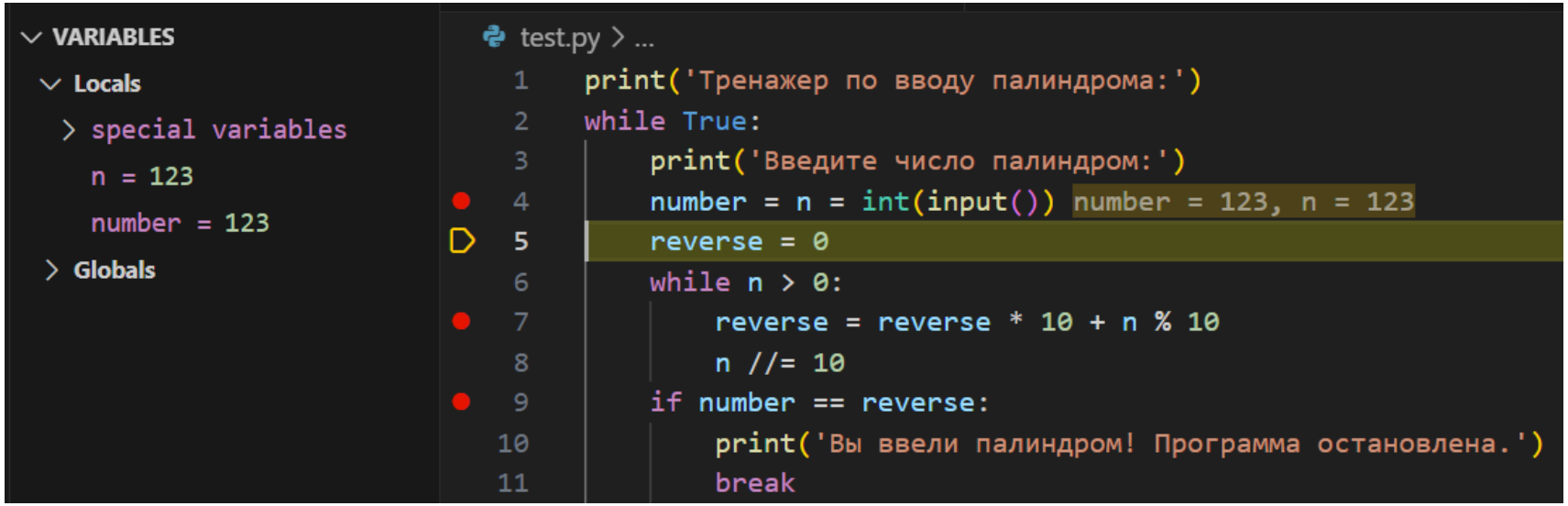
3. Вводим число:

Допустим, вводим 123.

- Программа останавливается на первом брейкпоинте: смотрим, что number = 123, n = 123.
- Шагаем (горячая клавиша F10) через внутренний цикл и смотрим, как меняются значения переменных:



Отладка по-взрослому



Шаг 1: reverse = 0 * 10 + 123 % 10 = 3, n = 123 // 10 = 12.
Шаг 2: reverse = 3 * 10 + 12 % 10 = 32, n = 12 // 10 = 1.
Шаг 3: reverse = 32 * 10 + 1 % 10 = 321, n = 1 // 10 = 0.
Цикл завершается. Смотрим: reverse = 321, number = 123 → условие не срабатывает.
Вывод: Всё работает, но палиндрома нет.

- Проверяем крайние случаи:
  Вводим 121 → дебаггер показывает, как reverse становится 121 → условие срабатывает.

- Вводим 5 → внутренний цикл не выполняется (n = 5 > 0 → выполняется один раз: reverse = 5, n = 0).
  Условие 5 == 5 → палиндром!

Итог: знаем о циклах всё (всё?)



Чисто теоретически с использованием текущих знаний вы можете решить любую задачу в мире*.

Вы узнали:

- Что break и continue — ваши союзники для управления ходом исполнения циклом, но важно использовать их с умом, чтобы не превратить программу в визуализацию теории заговора.
- Флаги делают код понятнее (но превращать код в ёлку из флажков тоже так себе идея).
- Вложенные циклы — мощь, которая требует аккуратности и убивает производительность, если входные данные очень большие. Если вложенность больше 3 — скорее всего, вы что-то делаете не так.
- Отладка позволяет избавиться от ошибок и снижает требования к разработчику в плане ясновидения.

* С некоторыми оговорками — вам иногда нужно будет заводить тысячи переменных, писать одни и те же куски кода много сотен раз (и много-много грустить).

P. S. Если вам кажется, что ваш код работает правильно с первого раза, то алгоритм действий такой:
1. Убедиться, что вы не спите.
2. Проверить программу на крайних случаях.
И только потом радоваться.